1. 问题： 哪个函数增长得更快：$NlogN$和$N^{1+\varepsilon/\sqrt{logN}}$，其中$\varepsilon > 0$，给出具体分析。

解析：令$log\ N\ = t\ \ N = e^t$

$$\lim_{N\to\infty}\frac{NlogN}{N^{1+\frac{\epsilon}{\sqrt{logN}}}} = \lim_{t\to\infty}\frac{e^t\cdot t}{(e^t)^{1+\frac{\epsilon}{\sqrt{t}}}} = \lim_{t\to\infty}\frac{t}{e^{\epsilon\sqrt{t}}} = \lim_{x\to\infty}\frac{x^2}{e^{\epsilon x}} = 0$$

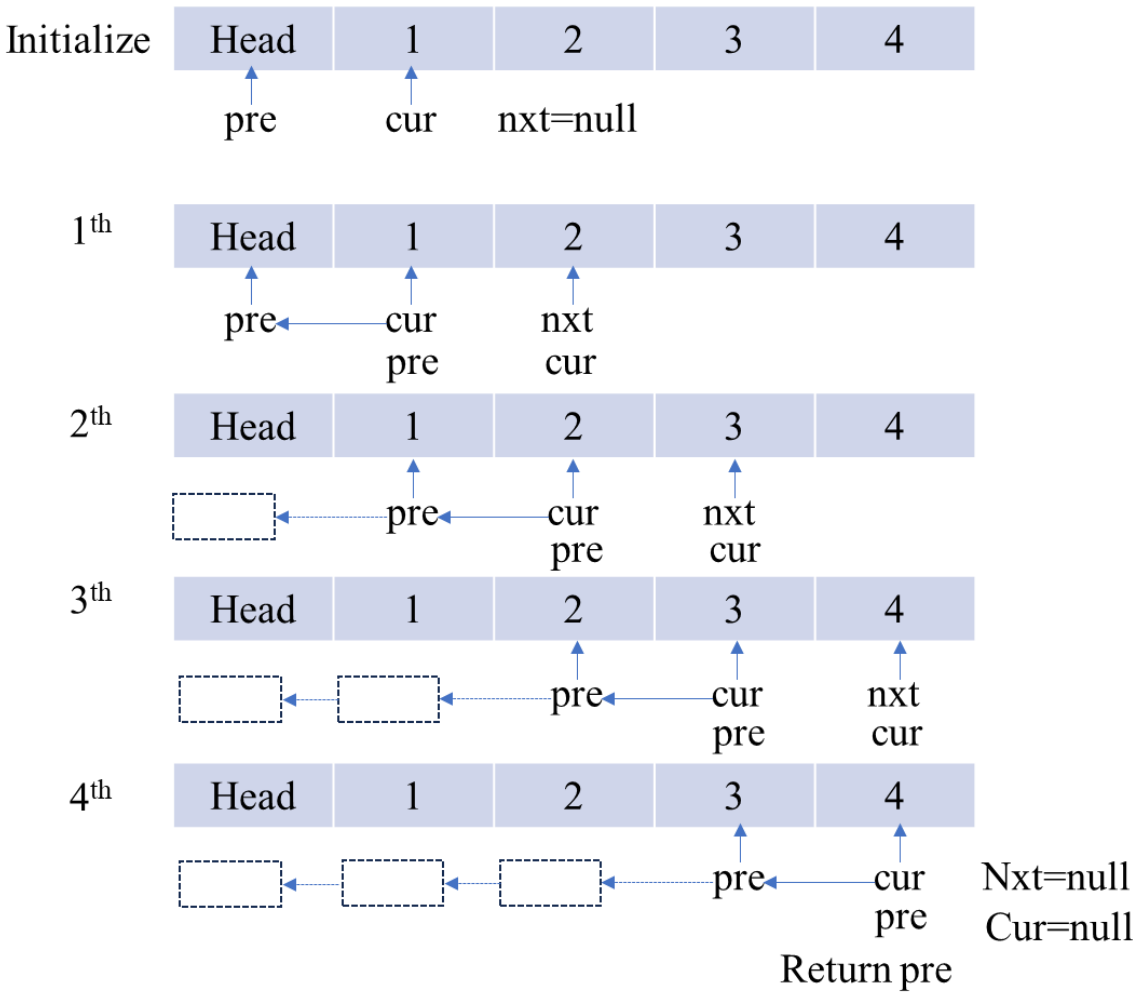
所以$N^{1+\frac{\epsilon}{\sqrt{logN}}}$增长更快。

2. 问题：考虑下述单向链表逆序的代码，针对链表长度为 4 的情况手绘草图分析指针的变化。考虑双向链表逆序的代码设计。

```
Node * reverseLinkList(Node * head) {
  if (head == NULL || head->next==NULL) return head;

  Node *pre = head;
  Node *cur = pre->next;
```

```
  Node *nxt = NULL;
  while (cur != NULL) {
    nxt = cur->next;
    cur->next = pre ;
    pre = cur;
    cur = nxt;
  }
  head->next = NULL;
  return pre;
}
```




```
Node* reverseDoublyLinkedList(Node* head) {
if (!head || !head->next) {
return head;
}

Node* prevNode = null;
Node* currNode = head;

while (currNode != null) {
// 交换节点的 prev 和 next 指针
Node* nextNode = currNode->next;
currNode->next = prevNode;
currNode->prev = nextNode;

// 移动到下一个节点
prevNode = currNode;
currNode = nextNode;
}
```

```
// 更新头节点
head = prevNode;

return head;
}
```

3. 考虑下述递归例程（图 1），注意 F(n)与 Fibonacci 存在差异。

（1）如果在 N=20 下运行且不考虑代码优化，请量化分析栈空间的使用情况。

（2）请将代码改写为循环的形式。

```
long F(long n){
  if(n<=3)
    return 1;
  else
    return F(n-1)+F(n-2)/4;
}
```

图 1 递归算法

设 F(n)的调用次数 c(n)

F(1), F(2),F(3)　0 次调用

F(4)　c(4)=2

F(5)　c(5) = c(4) + c(3) + 2 = 2 + 2 + 0 = 4

F(6)　c(6) = 2 + c(5) + c(4) = 2 + 4 + 2 = 8

c(n) = c(n-1) + c(n-2) + 2

c(7)　= 14　c(8) = 24　c(9) = 40 c(10) = 66 c(11) = 108　c(12) = 176

c(13) = 284 c(14) = 362 c(15) = 648 c(16) = 1012 c(17) = 1662 c(18) = 2676 c(19)=4340 c(20) = 7018

```
long F(float n) {
    if (n <= 3)
        return 1;

    float a = 1, b = 1;
    for (long i = 4; i <= n; ++i) {
        float temp = a + b / 4;
        a = b;
        b = temp;
    }
    return b;
}
```

4. 假设用 flag 作为区分循环队列空和非空的标志，设定 flag=0 为队列空，flag=1 为队列非空，初始 flag=0。请写出循环队列的入队与出队算法。当入队之前遇到队列满，以及出队之前遇到队列空时，需要打印警告。部分程序已给出，补齐循环队列的入队与出队算法。

```
template <class ElemType>
class CyQueue{
public:
    int front;  //queue[front]是头结点，不存数据
    int rear;
    int MaxSize;
    bool flag;
    ElemType *queue;
    CyQueue(int ms){
        flag=0;front=0;rear=0;
        MaxSize=ms;
        queue=new ElemType[ms+5]; //分配一个足够大的空间
    }
    void EnQueue(CyQueue &Q, ElemType x);
    ElemType OutQueue(CyQueue& Q);
};

template <class ElemType>
void CyQueue<ElemType>::EnQueue(CyQueue &Q, ElemType x)
{
```

【填空部分】

```
    If ((Q.rear == Q.front) && (Q.flag == 1){ cout<< “Queue Overflow” << endl;
      return;
    }
    Q.rear = (Q.rear + 1) % Q.MaxSize;
    Q.queue[Q.rear] = x;
    Q.flag = 1;}

template <class ElemType>
ElemType CyQueue<ElemType>::OutQueue(CyQueue &Q)
{
```

【填空部分】

```
 if(Q.flag == 0)
    { cout << “Queue empty” << endl;
     return;
    }
  Q.front = (Q.front +1) % Q.MaxSize;   if (Q.front == Q.rear) Q.flag = 0;
  return  Q.queue[Q.front];
}
```